

Module 04

PostgreSQL + Supabase

Relational databases, SQL, schema design, and the Supabase Python client

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is a Relational Database?

A relational database stores data in **tables** (like spreadsheets). Each table has named **columns** (schema) and **rows** (records). Tables can reference each other via **foreign keys**. SQL (Structured Query Language) is the universal language for querying and modifying this data.

PostgreSQL (Postgres) is the most powerful open-source relational database. It supports JSON columns, full-text search, geospatial queries, and extensions like pgvector for vector similarity search.

2. Core SQL Commands

```
-- Create a table
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,      -- auto-incrementing integer
  email       TEXT UNIQUE NOT NULL,
  name        TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Insert a row
INSERT INTO users (email, name) VALUES ('alice@example.com', 'Alice');

-- Read rows
SELECT id, email, name FROM users;
SELECT * FROM users WHERE email = 'alice@example.com';
SELECT * FROM users ORDER BY created_at DESC LIMIT 10;

-- Update a row
UPDATE users SET name = 'Alice Smith' WHERE id = 1;

-- Delete a row
DELETE FROM users WHERE id = 1;
```

3. Primary Keys, Foreign Keys, and Indexes

```
-- Primary key: unique identifier for each row
CREATE TABLE posts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT NOT NULL,
  body        TEXT,
  user_id     INTEGER NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- Index: makes lookups fast (like a book index)
CREATE INDEX idx_posts_user_id ON posts(user_id);

-- Without the index: Postgres scans every row (slow for large tables)
-- With the index: Postgres jumps directly to matching rows (fast)

-- Unique constraint: no two rows can have the same value
ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);
```

4. What is Supabase?

Supabase is a hosted Postgres database with extras built on top: REST API auto-generated from your schema, auth, realtime subscriptions, and storage. You get a fully managed Postgres database with a Python client that makes CRUD operations easy without writing raw SQL every time.

```
pip install supabase

from supabase import create_client, Client

SUPABASE_URL = "https://your-project.supabase.co"
SUPABASE_KEY = "your-service-role-key" # from Supabase dashboard

supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)
```

5. CRUD with the Supabase Python Client

```
# ■■■ CREATE (INSERT) ■■■
resp = supabase.table("users").insert({
    "email": "alice@example.com",
    "name": "Alice",
}).execute()
print(resp.data) # [{"id": "...", "email": "alice@...", "name": "Alice"}]

# ■■■ READ (SELECT) ■■■
resp = supabase.table("users").select("*").execute()
all_users = resp.data

resp = supabase.table("users").select("id, email").eq("name", "Alice").execute()
matching = resp.data

# ■■■ READ with ordering and limit ■■■
resp = (supabase.table("users")
        .select("*")
        .order("created_at", desc=True)
        .limit(10)
        .execute())

# ■■■ UPDATE ■■■
resp = (supabase.table("users")
        .update({"name": "Alice Smith"})
        .eq("email", "alice@example.com")
        .execute())

# ■■■ DELETE ■■■
resp = (supabase.table("users")
        .delete()
        .eq("email", "old_user@example.com")
        .execute())
```

6. Upsert — Insert or Update

Upsert inserts a row if it doesn't exist, or updates it if it does. This is critical for idempotent operations (running the same code twice gives the same result).

```
# Upsert: if the email already exists, update name; otherwise insert
resp = supabase.table("users").upsert({
    "email": "alice@example.com",
    "name": "Alice Updated",
}, on_conflict="email").execute()
```

7. Filtering — Where Clauses

```
# Equal
.eq("status", "active")

# Not equal
.neq("status", "deleted")

# In a list
.in_("id", ["id1", "id2", "id3"])

# Less than or equal
.lte("scheduled_at", datetime.utcnow().isoformat())

# Greater than
.gt("score", 4.0)

# Is null
.is_("deleted_at", "null")

# Full example: approved ideas for LinkedIn
resp = (supabase.table("ideas")
        .select("*")
        .eq("approval_status", "approved")
        .eq("platform", "linkedin")
        .order("score", desc=True)
        .limit(5)
        .execute())
```

8. Calling Stored Procedures with .rpc()

Supabase lets you define Postgres functions (stored procedures) that you can call via the Python client. This is used for vector similarity search with pgvector.

```
# Call a Postgres function: match_knowledge_base(query_embedding, match_count)
resp = supabase.rpc(
    "match_knowledge_base",
    {"query_embedding": [0.1, 0.2, ...], "match_count": 5}
).execute()
results = resp.data    # list of matching KB chunks with similarity scores
```

9. Schema Design — ContentAutomation2's 15 Tables

Good schema design follows these rules:

- Each table has one clear purpose (single responsibility)
- Use UUIDs as primary keys (not sequential integers) for distributed safety
- Store status as an enum column, not separate tables
- Use JSONB for flexible nested data (metrics, flags, summaries)
- Add indexes on columns you filter/sort on frequently
- Soft delete: add an 'active' or 'deleted_at' column instead of actually deleting

10. Practice Exercises

- Exercise 1: Create a Supabase project (free at supabase.com), get your URL and service key.
- Exercise 2: Create a 'tasks' table: id (uuid, pk), title (text), done (bool default false), created_at (timestampz).
- Exercise 3: Insert 5 tasks using the Python client.
- Exercise 4: Query all tasks where done=false, ordered by created_at.
- Exercise 5: Update task 1 to done=true.
- Exercise 6: Upsert a task by title — run it twice, verify no duplicates.
- Exercise 7: Design the schema for a blog: users, posts, comments. Draw the relationships.